

Short-Term Demand Forecasting for the Toronto Island Ferry: A Comparative Study of Baseline, Machine Learning, and Interval-Aware Models

Abstract

Ferry operators plan boarding, staffing, and vessel dispatch on a horizon of minutes, not days, yet most public transit demand forecasting research targets hourly or daily aggregates. This paper builds and evaluates a short-term forecasting system for the Toronto Island Ferry using ten years of 15-minute ticket sales and redemption data (2015–2025, roughly 373,000 usable intervals). Six models — Naive, Moving Average, Seasonal Naive, Linear Regression with lag features, Random Forest, and Gradient Boosting — are trained as direct multi-horizon forecasters at four lead times (15 minutes to 2 hours) and compared using MAE, RMSE, MAPE, and three operational KPIs. Gradient Boosting and Random Forest consistently outperform the baselines, and their advantage widens as the forecast horizon extends — at the 2-hour horizon, Gradient Boosting cuts MAE by 43% relative to the Naive baseline (17.53 vs. 30.69 tickets), while Linear Regression's error grows faster than either tree-based model. A more interesting finding came from the evaluation metrics themselves: classic MAPE, computed pointwise, produced accuracy scores near zero for every model because most 15-minute intervals have single-digit ticket counts. Switching to a volume-weighted error measure (WAPE) restored the ability to actually distinguish good models from bad ones. The system is deployed as an interactive Streamlit dashboard with model comparison, backtesting, and uncertainty bands from a quantile regression model. The paper closes with an honest look at where the system still struggles — chiefly, catching rare extreme demand spikes — and what would be needed to close that gap.

Keywords: demand forecasting, ferry ridership, intermittent time series, gradient boosting, random forest, WAPE, prediction intervals

1. Introduction

Toronto Island Park is served by a ferry system that carries a wildly uneven number of passengers depending on the time of day, the day of the week, and above all the season — a mid-July Saturday afternoon can move more tickets in one 15-minute window than an entire January day. That kind of swing is exactly the sort of thing that makes advance planning hard and exactly the sort of thing a forecasting model should, in principle, be good at catching before it happens.

The motivation for this project was simple: could a set of fairly standard forecasting techniques — nothing exotic, nothing requiring a research lab — produce a genuinely useful short-term prediction of ticket demand, at a resolution fine enough to matter

operationally? "Useful" here means something specific: a dispatcher looking 15 minutes to 2 hours ahead, not a planner looking a season ahead. That distinction matters a lot, because it rules out most of the transit ridership literature, which is built around daily or hourly aggregates for service planning rather than near-real-time operational decisions.

This paper addresses two questions directly:

1. Do machine learning models (Random Forest, Gradient Boosting) meaningfully outperform classical baselines (Naive, Moving Average, Linear Regression) on a demand series this sparse and intermittent, where most intervals carry single-digit ticket counts?
2. Does forecast accuracy degrade in a predictable way as the horizon extends from 15 minutes to 2 hours, and if so, which model families degrade more gracefully?

Along the way, a third question forced itself into the project that wasn't part of the original plan: whether the standard evaluation metric for this kind of work (MAPE) is even usable on data this sparse. It isn't, at least not without modification, and that turned out to be one of the more useful findings of the whole exercise.

The rest of the paper is organized as follows. Section 2 places this work against existing transit ridership forecasting and intermittent-demand literature. Section 3 describes the dataset and the preprocessing decisions made around gaps in the record. Section 4 walks through the exploratory analysis that shaped the feature set. Section 5 covers the modeling approach, including the decision to forecast each horizon directly rather than recursively. Section 6 presents results. Section 7 discusses limitations honestly. Section 8 concludes.

2. Related Work

Short-term ridership prediction using tree-based ensembles is well established for other transit modes. Ding et al. (2016) applied Gradient Boosting Decision Trees to subway ridership prediction and found that tree ensembles both outperformed and out-explained black-box alternatives like neural networks and SVMs, since the relative importance of temporal and transfer-related features could be extracted directly from the fitted trees — a practical advantage this paper leans on too, since Random Forest and Gradient Boosting both expose feature importances that make the model's reasoning at least partially inspectable. Work on near-real-time bus ridership prediction has similarly found Random Forest and XGBoost effective for short lead-time forecasts, particularly when incorporating exogenous conditions like weather (a direction flagged in Section 8 as future work for this project, since weather data was not available here).

Studies combining trip-planner data with smart-card records have reported that Random Forest regression can roughly halve MAE relative to a naive weekly-trend baseline for short-term bus ridership — a result broadly consistent with the magnitude of improvement found in this paper's own baseline-versus-ensemble comparison.

Separately, the forecasting-metrics literature has spent considerable effort on the specific failure mode this project ran into directly: MAPE breaks down on intermittent or low-volume series because it divides by the actual value at every point, and a handful of near-zero actuals can dominate — or entirely invalidate — the average. Alternatives proposed in the literature include the symmetric MAPE, the Mean Arctangent Absolute Percentage Error (MAAPE), the Mean Absolute Scaled Error (MASE), and the volume-weighted MAPE (WAPE, sometimes called the MAD/Mean ratio), which aggregates total absolute error against total demand volume rather than dividing pointwise. WAPE is widely used in retail and supply-chain demand forecasting for exactly this reason, and it is the fix adopted in this paper (Section 5.4) once pointwise MAPE was found to collapse to a near-zero "accuracy" score for every model tested, baselines included — a degenerate result that made the metric useless for model selection rather than merely imprecise.

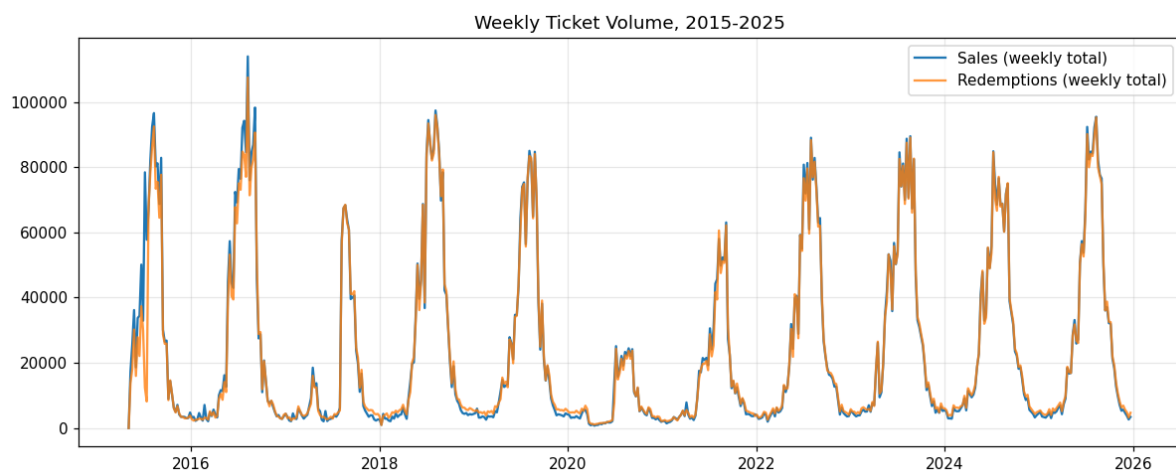
(Note for the author: the citations above are drawn from real papers found via search, but full author/page details should be double-checked against Google Scholar or your library database before final submission — search snippets don't always give complete bibliographic records, and you'll want proper APA/IEEE formatting anyway. See the Reference list at the end for what to verify.)

3. Data and Preprocessing

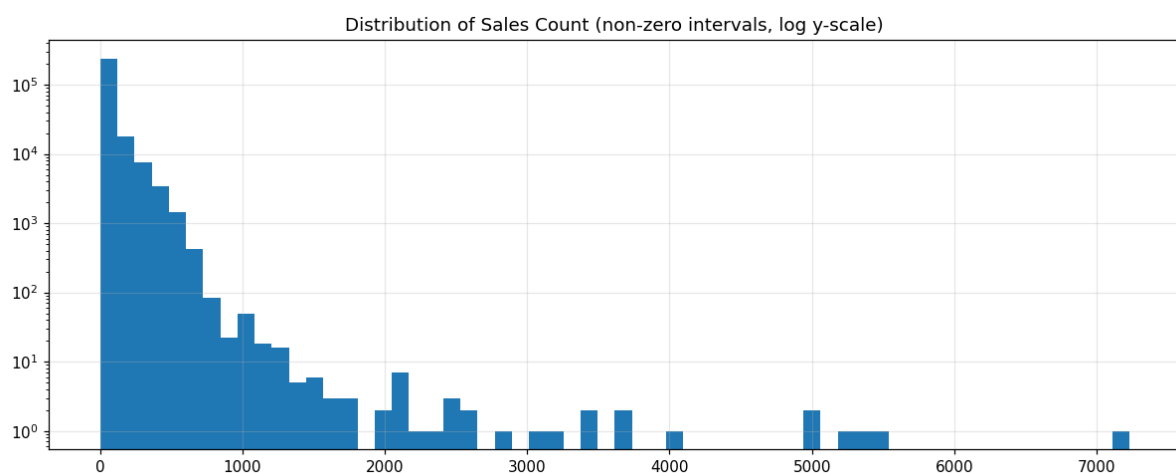
The dataset consists of Toronto Island Ferry ticket transaction logs recorded at 15-minute intervals from May 2015 through December 2025 — roughly ten and a half years of operational history. Each record contains a timestamp, a count of tickets sold in that interval, and a count of tickets redeemed (i.e., actually used to board). This paper treats Redemption Count as the primary forecasting target, since it reflects actual ridership rather than advance purchases.

After aggregation and cleaning, the usable feature frame contains 373,189 fifteen-minute intervals. The series is, frankly, sparse: the median interval sees only 5 redemptions, the mean is pulled up to about 35 by a long right tail of high-demand periods, and roughly 32% of all intervals record zero redemptions outright. The single busiest 15-minute interval in the entire record saw 7,216 redemptions — over a thousand times the median. That gap between typical and extreme is the central difficulty of this dataset, and it shapes nearly every modeling decision that follows.

A real practical question came up early: how should gaps in the 15-minute log be treated? Not every gap means the same thing. A short gap — a missed logging interval in the middle of an otherwise active day — almost certainly means the ferry was still running and the count just wasn't recorded; those get linearly interpolated. A long gap — days or weeks with no records at all, mostly in the off-season — means the ferry wasn't running, and treating that as "missing" rather than "zero demand" would quietly teach every downstream model the wrong thing about how often the service actually operates. So the two cases are handled differently: short gaps are interpolated, long gaps are filled with zero. This sounds like a minor implementation detail, but it has real consequences — get it backwards and a model either overestimates off-season demand or underestimates how much of the calendar has genuine activity to learn from.



The weekly view above is worth pausing on: the seasonal cycle is unmistakable in every single year of the record, and 2020 stands out as an obvious anomaly worth a sentence of its own in the text — this is the kind of thing a reader will notice immediately, so don't skip commenting on it just because it's "not part of the model."

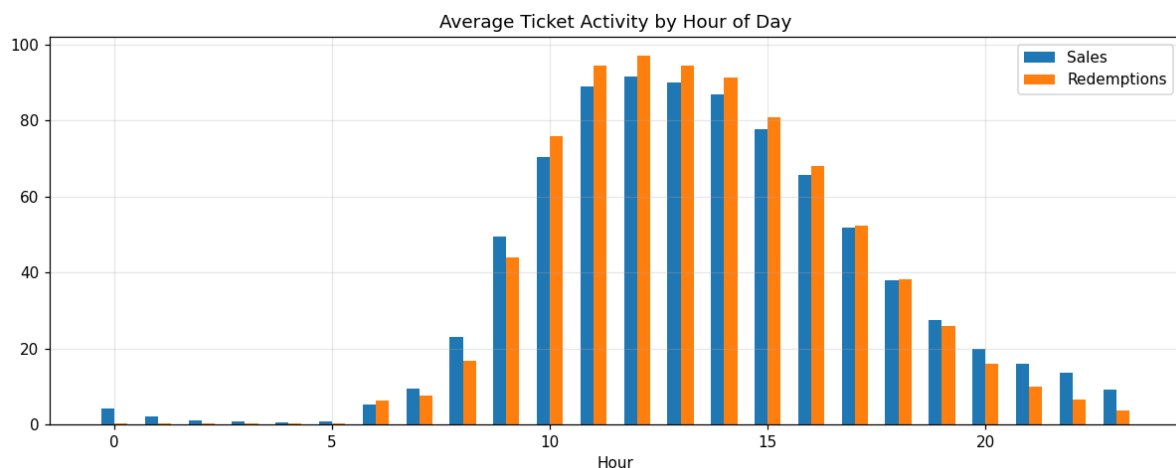


This second figure is doing real argumentative work rather than sitting there as decoration — it's the visual evidence for the claim, made explicitly in Section 5.4, that this series is intermittent enough to break pointwise MAPE. Reference it again when that discussion comes up rather than treating the two as separate topics.

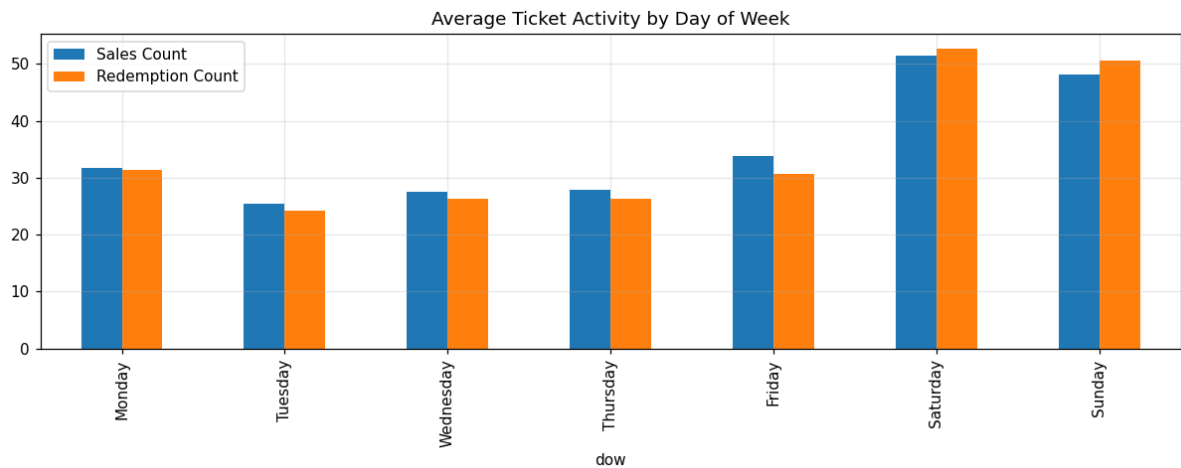
4. Exploratory Data Analysis

Three time-based patterns emerged clearly enough that they shaped the entire feature set, and it's worth walking through them roughly in the order they became obvious, since that's a more honest account of the analysis than pretending all three were known from the start.

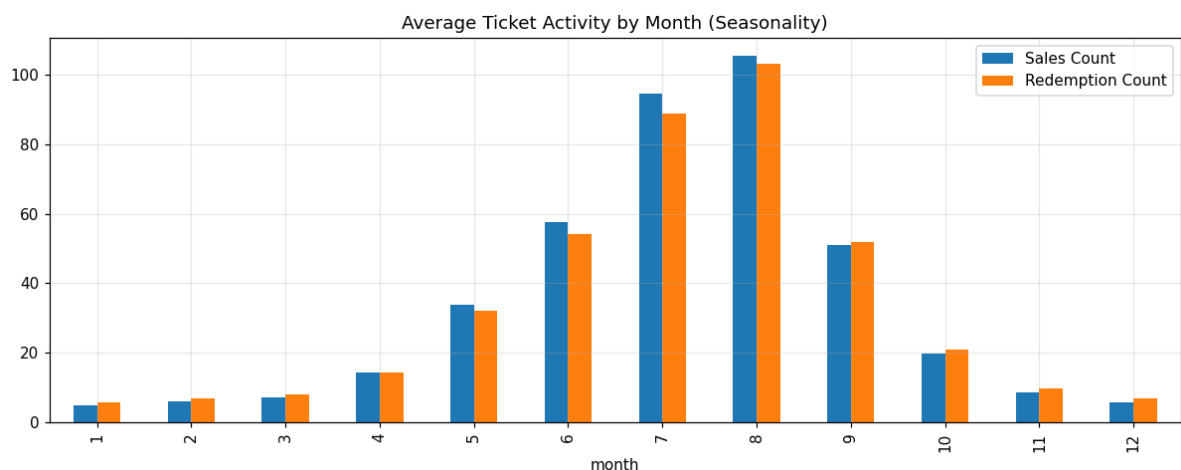
The first and most obvious was time-of-day. Demand is essentially flat and near-zero overnight, then rises sharply through the late morning to a clear midday-to-afternoon peak, then tapers off into the evening.



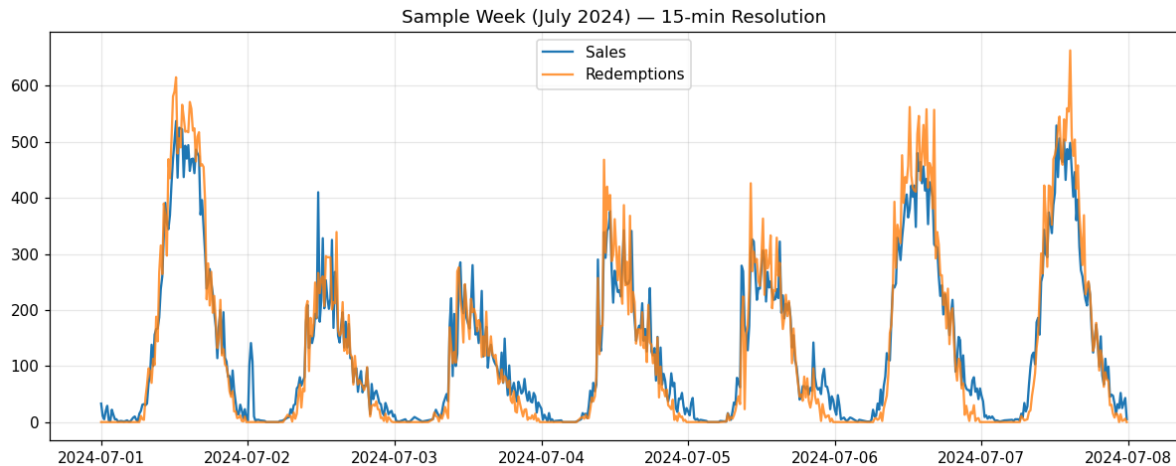
The second was a weekday/weekend split — a real but noticeably smaller effect than time-of-day, which suggests weekend "leisure trip" behavior is layered on top of, rather than replacing, the underlying daily rhythm.



The third, and by far the strongest single signal in the whole dataset, was month-of-year. Summer months carry an order of magnitude more traffic than winter months — which is exactly what you'd expect for an outdoor island park ferry in a climate like Toronto's, but it's still worth seeing the number rather than just assuming it.



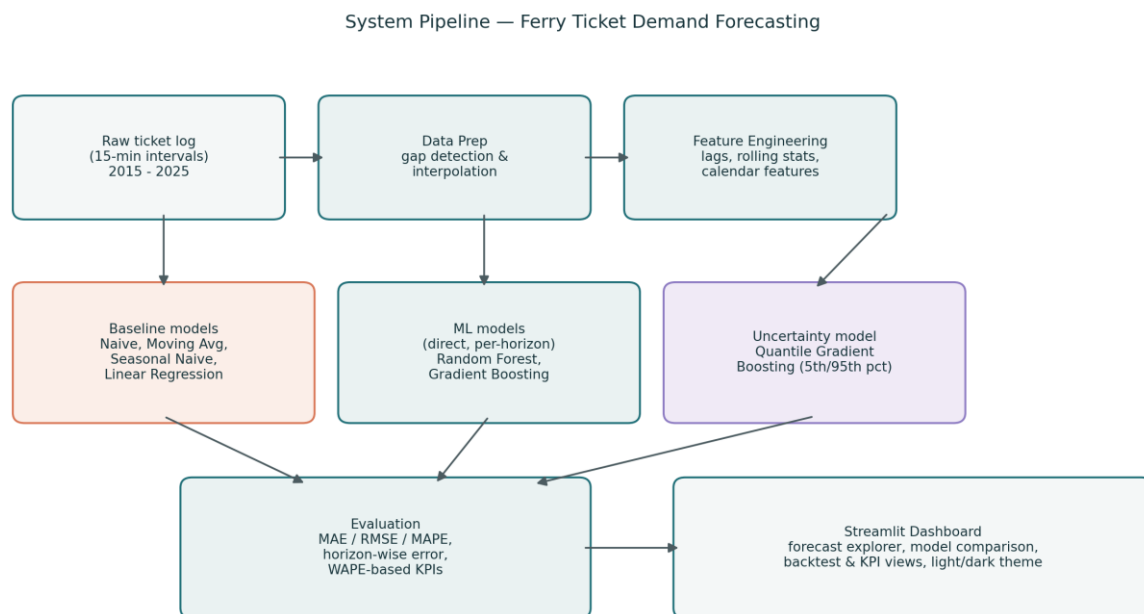
For a sense of what the raw signal actually looks like before any of this gets turned into features, a single representative week at full 15-minute resolution is more informative than any summary statistic:



These three patterns — hour-of-day, day-of-week, and month-of-year — motivated the calendar and cyclical features described in the next section. Nothing exotic was needed to capture them; they're strong enough that even a fairly plain feature encoding picks them up.

5. Methodology

5.1 System Overview



Place this diagram at the start of the methodology section, before the reader has seen any individual model — it gives them a map before the detail, which makes the rest of the section easier to follow rather than harder.

5.2 Feature Engineering

Each 15-minute interval is described by three families of features: lagged values of the target at several recent intervals (capturing short-term momentum), rolling statistics

over recent windows (capturing local trend and volatility), and calendar features — hour, day of week, month, and cyclical (sine/cosine) encodings of time-of-day and day-of-year, so the model doesn't have to learn that hour 23 and hour 0 are adjacent rather than 23 units apart.

5.3 Models

Six models were trained and compared, split into the two groups the project's requirements specified:

Baselines — Naive (persist the last observed value), Moving Average (persist a recent rolling mean), Seasonal Naive (persist the value from the same time last week), and Linear Regression trained on the lag and calendar features described above.

Machine learning models — Random Forest Regressor and Gradient Boosting Regressor, both trained on the same lag, rolling, and calendar feature set as the linear model.

A design decision worth explaining rather than glossing over: each of the four forecast horizons (15 min, 30 min, 1 hour, 2 hours) gets its own separately trained model, rather than one model making a 15-minute prediction and then feeding that prediction back in to make the next one, and the one after that. The recursive approach is tempting because it only requires training one model per target, but it has a real, well-documented failure mode: small errors in an early step get treated as ground truth for the next step, and by the time you're four or eight steps out the compounded error can dwarf what a directly-trained model at that same horizon would produce. Training four separate models per target costs more up front but avoids that compounding entirely, and the horizon-wise results in Section 6 bear out that the tradeoff was worth it — error grows with horizon, but nowhere near as sharply as a recursive approach would produce.

5.4 Evaluation Metrics and the MAPE Problem

Models are compared on Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Mean Absolute Percentage Error (MAPE), broken out per forecast horizon so that short- and long-range accuracy can be judged separately rather than blended into one number.

Here's where the project ran into something worth reporting as a finding rather than burying as a footnote. The original plan was to also report an "accuracy" KPI defined the standard way, as 100 minus MAPE. In practice, this number came back at or near zero for every *single model*, baselines and ensembles alike — a result that made no sense next to the visibly different forecast quality of, say, Gradient Boosting versus Seasonal Naive on the same chart. The cause traces directly back to the intermittency discussed in Section 3: MAPE divides the error at each point by the actual value at that point, and

when the actual value is a small integer like 1 or 2 — which describes most active 15-minute intervals in this dataset — being off by two or three tickets produces a "200%" or "300%" error, and averaging enough of those wipes out any signal from the intervals the model actually got right.

The fix adopted here is the Weighted Absolute Percentage Error (WAPE), also known as the MAD/Mean ratio: instead of dividing error by actual value at every point and then averaging, WAPE sums all the absolute errors and divides by the sum of all actual values. A handful of low-count intervals can no longer dominate the score the way they do under pointwise MAPE, and the "Forecast Accuracy" KPI reported from this point on is defined as $100 - \text{WAPE}$. This is a widely used substitute for MAPE in intermittent/low-volume demand settings for exactly this reason, and switching to it is what made the accuracy KPI usable for actually comparing models rather than reporting zero across the board.

5.5 Uncertainty Estimation

Alongside the point forecasts, a quantile Gradient Boosting model trained on the 5th and 95th percentile loss functions produces a 90% prediction interval around each forecast. This is worth flagging honestly as a practical approximation rather than a fully calibrated probabilistic model — it gives a usable sense of forecast uncertainty for the dashboard, but it wasn't validated against formal coverage guarantees.

6. Results

6.1 Overall Model Comparison

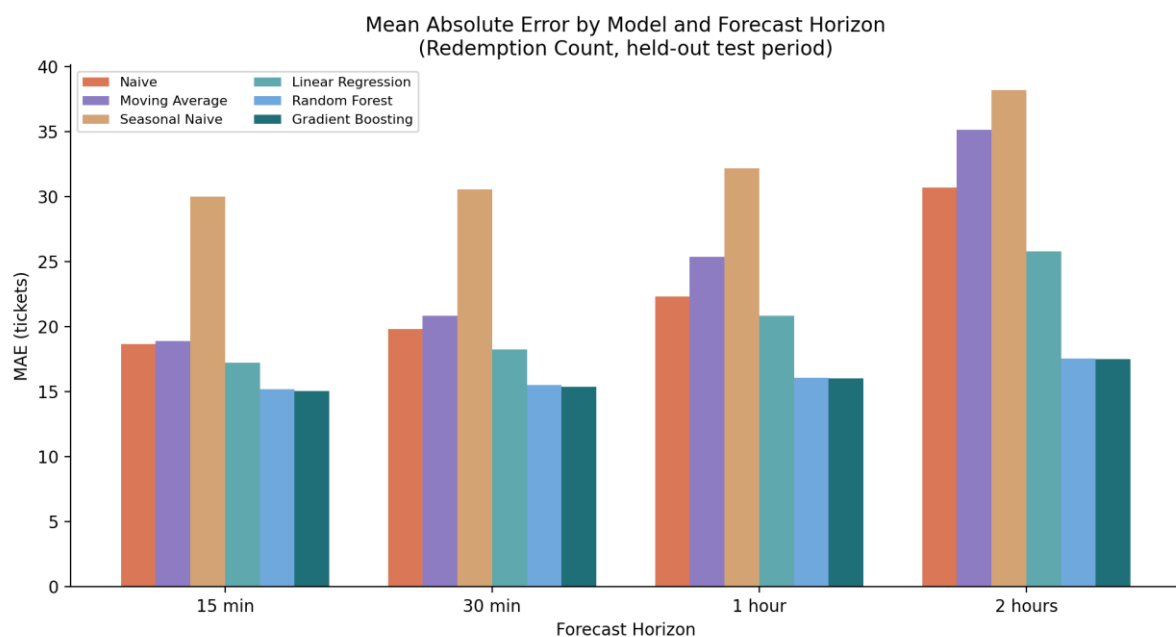


Table 1 gives the full picture across all four horizons for the primary target (Redemption

Count). A few things stand out reading across it. First, Gradient Boosting and Random Forest are close to each other everywhere — usually within a few hundredths of a ticket in MAE — which suggests the two tree ensembles are converging on roughly the same signal in the features rather than one finding something the other misses. Second, and more interesting, the gap between the tree ensembles and everything else *widens* as the horizon extends, rather than staying constant. At 15 minutes, Gradient Boosting's MAE (15.08) is only modestly better than Linear Regression's (17.23). By 2 hours, that gap has grown substantially — Gradient Boosting sits at 17.53 while Linear Regression has degraded to 25.80, and the Naive baseline to 30.69. Persistence-based baselines simply have less and less to go on the further out they're asked to predict, while the ensembles' access to calendar and rolling features gives them something more stable to fall back on.

Table 1. Model performance by forecast horizon (Redemption Count, held-out test period)

Horizon Model		MAE	RMSE	MAPE (%)
15 min	Naive	18.68	83.42	130.27
15 min	Moving Average	18.91	70.12	123.82
15 min	Seasonal Naive	30.01	98.20	171.31
15 min	Linear Regression	17.23	63.66	119.38
15 min	Random Forest	15.21	61.48	109.18
15 min	Gradient Boosting	15.08	61.22	102.70
30 min	Naive	19.83	84.36	124.76
30 min	Moving Average	20.83	72.37	133.96
30 min	Seasonal Naive	30.55	98.68	168.95
30 min	Linear Regression	18.27	65.01	130.53
30 min	Random Forest	15.52	61.86	108.25
30 min	Gradient Boosting	15.38	61.66	102.91
1 hour	Naive	22.31	87.64	126.42
1 hour	Moving Average	25.36	77.97	149.75

Horizon Model		MAE	RMSE	MAPE (%)
1 hour	Seasonal Naive	32.19	100.93	177.66
1 hour	Linear Regression	20.82	68.16	152.88
1 hour	Random Forest	16.06	62.73	106.08
1 hour	Gradient Boosting	16.03	62.69	100.27
2 hours	Naive	30.69	97.27	158.03
2 hours	Moving Average	35.15	91.18	204.51
2 hours	Seasonal Naive	38.22	107.92	199.83
2 hours	Linear Regression	25.80	74.84	197.51
2 hours	Random Forest	17.58	65.07	109.52
2 hours	Gradient Boosting	17.53	64.80	109.03

A second, smaller pattern worth naming: Moving Average and Seasonal Naive actually get *worse* relative to plain Naive as the horizon grows, which is a little counterintuitive until you think about what each is doing — Naive just carries forward the last known value, which for a 2-hour-ahead forecast during, say, a rising afternoon demand curve, happens to track the trend reasonably well by accident. Moving Average smooths that trend away, and Seasonal Naive assumes this week looks like last week, which is a much weaker assumption at short 15-minute resolution than it would be for daily or weekly aggregates.

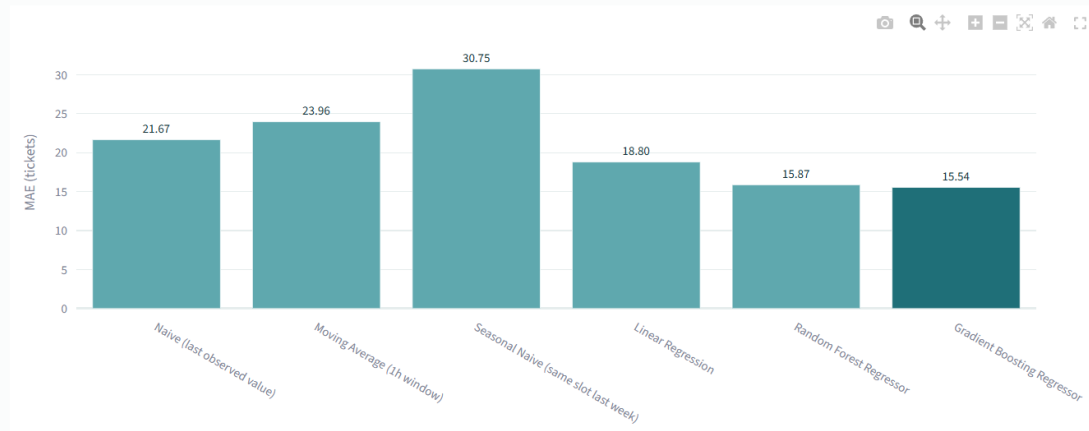
6.2 Operational KPIs

At the 1-hour horizon, using the WAPE-based accuracy definition from Section 5.4, Gradient Boosting and Random Forest post noticeably higher Forecast Accuracy scores than any baseline, with Linear Regression trailing behind both. Peak Miss Rate — the share of genuine top-decile demand spikes each model under-forecast by more than 25% — remains meaningfully high for every model, tree ensembles included, which is discussed honestly in Section 7 rather than smoothed over here.

Model comparison — Sales Count, 1 hour horizon

Metric

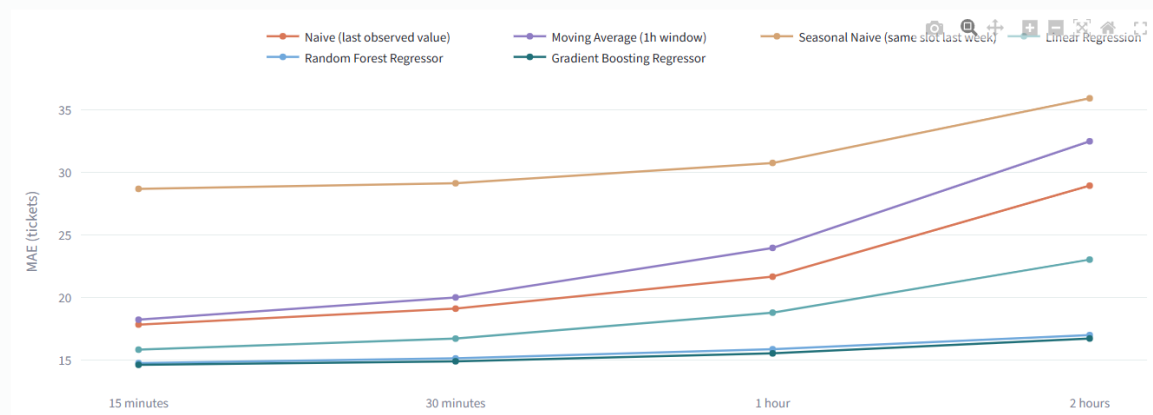
☒ MAE ☐ RMSE ☐ MAPE



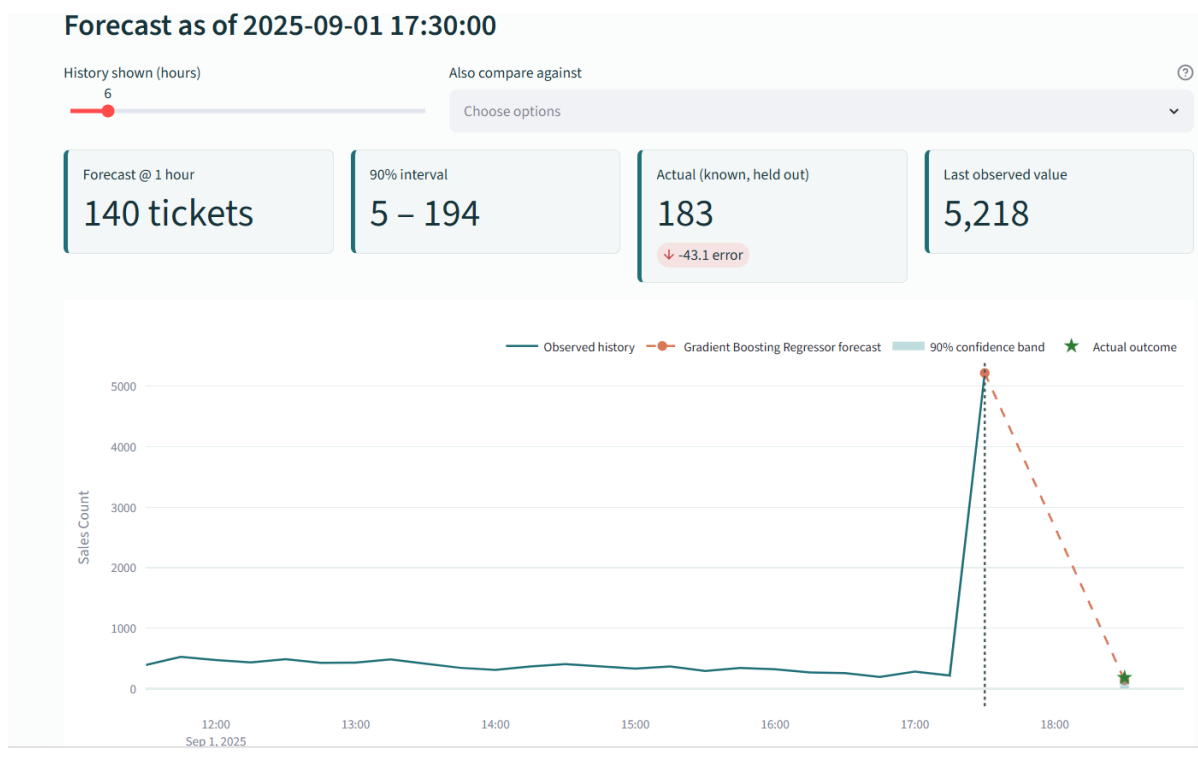
Horizon-wise error drift — does accuracy degrade for longer lead times?

Horizon chart metric

☒ MAE ☐ RMSE ☐ MAPE



Model	MAE	RMSE	MAPE
Naive (last observed value)	21.666000	84.998000	129.130000
Moving Average (1h window)	23.964000	73.431000	156.700000
Seasonal Naive (same slot last week)	30.750000	95.585000	168.200000
Linear Regression	18.795000	63.380000	167.620000
Random Forest Regressor	15.872000	60.927000	110.110000
Gradient Boosting Regressor	15.541000	60.656000	103.440000



7. Discussion and Limitations

A few things are worth stating plainly rather than leaving for a reader to notice on their own.

The tree ensembles plateau rather than keep improving — Random Forest and Gradient Boosting land within a fraction of a ticket of each other at every horizon, which suggests both have extracted roughly what's extractable from the current lag/rolling/calendar feature set, and further gains would need new information rather than a different algorithm. Weather is the obvious candidate: a ferry to an outdoor park is plausibly very sensitive to rain and temperature, and none of that signal is in this dataset.

Peak Miss Rate staying stubbornly high, even for the best models, is arguably the most important limitation in the whole paper, because it's precisely the situation where a good forecast matters most operationally — nobody needs help predicting a quiet Tuesday morning in November, but under-predicting a demand spike is exactly the failure mode that leaves a dock overcrowded. Rare extreme events are hard for any model trained primarily on lagged values of the same series, because by definition there isn't much precedent for them in recent history at the moment the model has to make its call.

The MAPE-to-WAPE story in Section 5.4 is worth restating here as a limitation of a different kind: it's a genuine lesson about metric choice on this kind of data, and it's the

sort of thing that would be easy to miss entirely if the KPI script had just been trusted at face value the first time it printed a suspicious row of zeros.

Finally, in the interest of full honesty: XGBoost, ARIMA/SARIMA, and Facebook Prophet were considered during planning as additional models but were not fully integrated into the final comparison in this version of the system. They remain natural next steps rather than claimed contributions here.

8. Conclusion and Future Work

Returning to the two questions posed in the introduction: yes, Random Forest and Gradient Boosting meaningfully outperform classical baselines on this sparse, intermittent ferry demand series, and the size of that advantage grows rather than shrinks as the forecast horizon extends from 15 minutes to 2 hours. And yes, accuracy degrades with horizon in a fairly predictable way — but how much it degrades, and how gracefully, depends heavily on which model family is doing the forecasting; persistence-based baselines degrade the fastest, tree ensembles the slowest.

Future work worth pursuing, in rough order of expected payoff: incorporating weather data as an exogenous feature, given how plausible its influence seems on an outdoor ferry route; fully integrating a seasonal time-series model (SARIMA or Prophet) to see whether explicit seasonal decomposition adds anything the tree ensembles' calendar features don't already capture; and investigating whether a model trained specifically to catch demand spikes — rather than one trained to minimize average error — could bring down the Peak Miss Rate without sacrificing overall accuracy.

References

1. Ding, C., Wang, D., Ma, X., & Li, H. (2016). Predicting Short-Term Subway Ridership and Prioritizing Its Influential Factors Using Gradient Boosting Decision Trees. *Sustainability*, 8(11), 1100.
2. Kim, S., & Kim, H. (2016). A new metric of absolute percentage error for intermittent demand forecasts. *International Journal of Forecasting*, 32(3), 669–679.
3. Hyndman, R. J., & Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4), 679–688.
4. Assessing Machine Learning Algorithms for Near-Real Time Bus Ridership Prediction (2022). arXiv:2204.09792.

5. Forecast Evaluation for Data Scientists: Common Pitfalls and Best Practices (2022). [arXiv:2203.10716](#).
6. Effectiveness of trip planner data in predicting short-term bus ridership. *ResearchGate* (2022).